SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I M. SANTOSH (192521007) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

Answer:

Behaviour. Greedy Best-First Search (GBFS) expands the node with the lowest heuristic value, f(n) = h(n), where h(n) is the straight-line distance to the delivery point. The drone therefore always heads for whichever neighbouring waypoint looks closest to the target and ignores the terrain and weather cost already incurred.

- **Strengths:** very fast, with low time and memory use in practice; it needs no cost bookkeeping, so it gives an immediate first route, which suits an urgent mission. It can re-plan cheaply when a path is reported blocked, because it simply picks the next-best-looking node.
- **Weaknesses under uncertainty:** it is neither optimal nor reliably complete (it can loop in graph search without a visited set). Straight-line distance ignores flooded terrain, wind and rain, so it may pick a short but dangerous or expensive route. It can be trapped in dead ends (a blocked road or a river between the drone and the goal looks "close" yet cannot be crossed), and it reacts badly to wrong or partial information because it has no measure of the real cost so far.

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

Answer:

Behaviour. A* ranks nodes by f(n) = g(n) + h(n). Here g(n) is the actual cost travelled so far, including terrain and weather-dependent edge costs, and h(n) is the straight-line distance still to go.

- **Balance:** g(n) keeps the search honest by penalising expensive, risky legs, while h(n) pulls the search towards the goal. When h = 0 A* behaves like uniform-cost search (careful but slow); when g is ignored it behaves like GBFS (fast but reckless). Combining them focuses the search yet keeps costs in view.
- **Guarantees:** straight-line distance never overestimates the true travel cost, so it is admissible (and consistent), and A* returns an optimal least-cost route and is complete.
- **Dynamic handling:** when a road is blocked the edge cost is set to infinity (or the edge is removed) and A* re-runs from the drone's current position. Higher cost terrain and bad weather are absorbed directly into g(n), so the route automatically avoids risky regions.

- **Cost:** it stores many nodes and is slower than GBFS, but this is acceptable with a good heuristic.

iii. Analyze the impact of heuristic accuracy on both algorithms

Answer:

Impact of heuristic accuracy.

- **GBFS:** its whole decision depends on h(n). A highly accurate heuristic sends it almost directly to the goal. An inaccurate one (straight-line distance in a flood zone, where the real path must detour) leads it into dead ends and poor routes with no way to correct itself, because it never measures real cost. It is therefore very sensitive to heuristic quality and its solution quality is unbounded.
- **A*:** if h is admissible and consistent, optimality is preserved however weak h is; accuracy affects only efficiency. A heuristic close to the true cost h*(n) expands very few nodes; a poor one (h close to 0) forces A* to expand many nodes, approaching uniform-cost search. If h overestimates (for example the heuristic is inflated to be faster), A* may lose optimality.
- **Under changing weather,** the straight-line distance becomes more optimistic (underestimates the true cost), which is safe for A* but misleading for GBFS. A better heuristic can include terrain and weather factors, as long as it never exceeds the true cost.

iv. Discuss how dynamic changes (blocked paths) affect search performance

Answer:

Effect of dynamic changes (blocked paths).

- When a path becomes blocked the previously computed route is invalid and the drone must re-plan. The search must treat the map as changing, so cached results (open/closed lists) become stale.
- **GBFS** recovers quickly because re-planning is cheap, but it may repeatedly walk into newly blocked dead ends and oscillate, with no guarantee the new route is good.
- **A*** returns an optimal route for the current map but each re-plan from scratch costs time. Incremental variants such as **D* Lite / LPA*** or **Anytime Repairing A*** reuse earlier search effort and only repair the affected part of the search, so re-planning is much faster.
- Partial real-time updates mean the drone plans with an incomplete map, so it should plan on the best known map, execute a few steps, then re-plan (receding-horizon planning). More frequent blockages raise the number of re-plans, and therefore delay and energy use. Keeping a safety margin (reserve battery, alternative routes) limits the risk.

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

Answer:

Recommendation: A* (preferably with incremental re-planning such as D* Lite and a weighted heuristic when time is critical).

- **Real-time constraints:** GBFS is fastest, but the search space (a graph of waypoints) is small enough for A* to run in milliseconds, and weighted A* ($f = g + w \cdot h$, w slightly above 1) or anytime A* can trade a little optimality for speed. D* Lite reuses previous work after each blockage.
- **Optimality:** A* with an admissible heuristic gives the minimum-cost route, which means the least delay and fuel consumption; GBFS can return arbitrarily long routes.
- **Safety:** because terrain and weather risk are encoded in g(n) (high cost or infinite cost for dangerous or blocked zones), A* avoids hazards explicitly, whereas GBFS ignores them.

Medicines carried by a drone also need a battery reserve, which A* can track as part of the cost.

- **Practical design:** use GBFS only to produce a quick initial fallback route, then let A*/D* Lite refine it as new information arrives. Overall, A* gives the best balance between speed, optimality and safety for this scenario.

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

Answer:

Optimisation formulation.

- **State:** the current timing plan, i.e. for every intersection i the cycle length, green-time split per phase and offset relative to neighbouring signals.
- **Objective function:** minimise F(x) = w1·(total waiting time) + w2·(fuel consumption) + w3·(congestion, e.g. queue length or density), with weights w1, w2, w3 chosen by city policy.
- **Constraints:** minimum and maximum green time, pedestrian crossing time, safety clearance (yellow/all-red), and fixed cycle limits.
- **Neighbourhood / operators:** change one signal's green time or offset by a small step. The objective is evaluated by simulation or from live traffic counts.

Why traditional (uninformed / exhaustive) search is inefficient. With hundreds of intersections each having many timing choices, the search space grows exponentially (about k^n for k settings and n intersections), so BFS, DFS and even A* cannot enumerate or store it. We care about a good configuration, not a path, so there is no explicit goal state or path cost to exploit. Traffic changes continuously, so any solution found quickly becomes outdated, and the objective is non-linear and noisy. Local-search and metaheuristic methods that keep only the current state (or a small population) and need very little memory are the right tools.

i. Explain how Hill Climbing would work in this scenario and identify its limitations

Answer:

Hill climbing. It starts from the current signal plan, generates neighbouring plans (adjust one intersection's green time or offset), evaluates the objective, and moves to the neighbour that improves it most (steepest-ascent) or to the first better neighbour (simple hill climbing). It stops when no neighbour is better. It uses very little memory, is fast and works online, so it can keep adjusting signals as traffic changes.

Limitations.

- It stops at a local optimum that is not the global best.
- It can stall on flat regions, and it cannot make a temporary move that gets worse to reach a better region.
- The result depends on the starting configuration, it is greedy, and it has no guarantee of optimality or completeness.
- With noisy traffic measurements it may chase noise.

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

Answer:

- **Local maximum:** a timing plan where every small change (lengthening or shortening any single green phase) makes things worse, yet a much better global plan exists. Example: a main road is optimised locally so that it flows well, but changing several junctions together (a "green wave") would give far lower total waiting time.
- **Plateau:** a region where many neighbouring plans give the same objective value, for example slightly different green times when traffic is light, so there is no gradient to follow and the search wanders or stops. A shoulder is a plateau that eventually leads uphill, so a limited number of sideways moves can help.
- **Ridge:** a narrow path of improvement that no single-step move follows, because several signals must change together. Example: improvement requires changing the offsets of two adjacent junctions at once to synchronise them; changing either alone makes waiting worse, so hill climbing cannot move along the ridge.

Remedies include random restarts, sideways moves, allowing compound moves (changing several signals together), and moving to stochastic methods.

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

Answer:

Simulated Annealing (SA). SA picks a random neighbour and always accepts an improvement. A worse move that increases the cost by Δ is accepted with probability e^(−Δ/T), where the temperature T starts high and is lowered gradually (cooling schedule). Early on the search explores widely and can escape local maxima, plateaus and ridges; as T falls it behaves more like hill climbing and settles into a good solution. With a slow enough cooling schedule it converges to the global optimum in theory. It is simple, uses little memory and works with one configuration.

Genetic Algorithm (GA). A population of candidate timing plans (chromosomes encode green times and offsets for all intersections) evolves through generations.

- **Fitness:** the weighted objective (waiting time, fuel, congestion) computed by simulation.
- **Selection** keeps good plans; **crossover** combines good parts of two plans (for example the timings of different districts); **mutation** randomly perturbs a gene to add diversity.
- Because it searches many points in parallel and recombines building blocks, it explores globally, handles ridges (crossover changes multiple signals at once) and escapes local optima. It is also parallelisable and easily keeps several different good plans.

iv. Recommend the best approach and justify your choice based on scalability and adaptability

Answer:

Recommendation: a hybrid approach, a Genetic Algorithm (or population-based search) for global exploration with Simulated Annealing / hill-climbing for local refinement and online adaptation.

- **Scalability:** GA evaluates populations in parallel across city zones (decomposing hundreds of intersections into districts), and chromosome encoding scales with the number of signals. SA alone is sequential and slower to converge for such a huge space, though it is lighter in memory.

- **Adaptability:** run the GA offline or at off-peak intervals to find good base plans for typical patterns (morning peak, evening peak, events). During operation, use SA or hill climbing with small moves, warm-started from the current plan, to adjust to real-time traffic. Keep part of the previous population when traffic changes so the search adapts rather than restarting.
- **Robustness:** a population and the probabilistic acceptance rule tolerate noisy sensor data and avoid local optima, which plain hill climbing cannot. Constraints (minimum green, pedestrian safety) are enforced as hard limits or penalties.

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

Answer:

Why an online search agent is needed.

- An **offline** search agent computes a complete plan before acting, which requires a known model of the environment (a full map, known actions and outcomes). The rover explores unknown terrain, so no complete map exists to plan on in advance.
- An **online** agent interleaves planning and acting: it takes an action, observes the new percepts, and then decides the next step. This is the only possible approach when the environment is unknown, since the agent must explore to discover the states, actions and costs.
- Mars conditions make offline planning impractical anyway: communication delay with Earth is several minutes, so remote control in real time is impossible, and the terrain (craters, rocks, dust) can be discovered only through sensors. Unexpected hazards need immediate reaction, and long plans would be invalid after the first surprise. Online search also limits the damage of wrong assumptions, because the rover commits only to one action at a time.

ii. Analyze the challenges of partial observability and dynamic environments

Answer:

Challenges.

- **Partial observability:** limited sensors (cameras, lidar with limited range, noise, shadows and dust) give only local, uncertain information. The rover cannot see beyond its range, so the true state of the world is hidden and different real states look identical (perceptual aliasing). It must keep a belief state, and noise can make a rock look like a safe slope or the reverse.
- **Dynamic environment:** conditions change (dust storms, shifting sand, lighting, battery and temperature), so earlier observations can become outdated and old plans fail.
- **Consequences:** actions are irreversible (a rover stuck in a crater cannot be rescued), so mistakes are costly; the agent may revisit the same places and loop, and exploration must be balanced with exploitation (collecting samples). Computation, memory, power and communication are limited, and safety margins must account for sensor uncertainty.

iii. Describe how the rover builds and updates its internal model

Answer:

Building and updating the internal model.

- The rover keeps a **map / model** as a data structure, such as an occupancy grid or a graph of visited positions, storing for each cell: unknown / free / obstacle / hazard, traversal cost, and optionally elevation and sample locations (a result table such as RESULT[s, a] = s′ in online-search terminology).
- **Sensing:** at each step it observes its surroundings with its sensors and localises itself (odometry, visual landmarks, SLAM).
- **Update:** newly seen cells are added or revised; new obstacles are marked and costs updated (e.g. a slope found steeper than expected). Probabilistic updates (Bayesian / belief update, for example occupancy probabilities) handle sensor noise.
- **Use:** after each update, the rover re-plans on the current model (for example with A* or D* Lite over the known part, treating unknown cells optimistically or pessimistically, or going to the nearest frontier), executes the first step(s), and repeats. For each state it stores which actions it has already tried and where it has been, so that it can backtrack and avoid infinite loops. Learning real-time A* (LRTA*) updates the heuristic value of visited states as it learns.

iv. Compare online search with uninformed and informed search strategies

Answer:

Comparison.

- **Uninformed (BFS, DFS, uniform-cost):** these need no domain knowledge, but they assume a known state space and expand states without direction. For a rover, systematic exploration wastes energy and time, and the physical agent cannot "jump" to another node: backtracking requires actual travel. DFS in an online setting can wander far, and BFS cannot be done without physically visiting nodes.
- **Informed (greedy, A*):** these use a heuristic (distance to the goal or sample site) to be much more efficient, but classically they are offline: they assume the whole graph is known, and they can jump to any node on the frontier in simulation. In a partly known map they plan on an incomplete model and need re-planning when it changes.
- **Online search (online DFS, LRTA*, D* Lite):** expands only the node the agent currently occupies, so every action is a real move. It handles unknown maps and changing percepts, keeps learning, and guarantees at least exploration (online DFS is complete in finite safe spaces). Its drawbacks are that the competitive ratio can be poor (the path is longer than the optimal one), and it is not optimal, and it is hard when actions are irreversible (dead ends).
- **Summary:** uninformed and informed offline methods assume knowledge and ignore the cost of physical movement; online search is designed for unknown environments, and can be combined with heuristics to get the efficiency of informed search.

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

Answer:

Recommended approach: an online, heuristic-guided search with continuous re-planning (e.g. D* Lite or LRTA*-style real-time A*) on a probabilistic map, with explicit safety constraints.

- **Uncertainty:** maintains a probabilistic map and belief about hazards; unknown cells are given conservative costs; the plan is recomputed as new data arrives.
- **Adaptability:** D* Lite reuses previous search effort and repairs the path efficiently when a new crater or rock is discovered; the heuristic (distance to the sample site or frontier) keeps exploration directed and energy-efficient.

- **Safety:** hazardous and uncertain cells get very high or infinite cost; the rover keeps a safety margin, prefers reversible moves (so it can always retreat), keeps enough battery to return, and stops and waits for the ground team when uncertainty is too high. Limited look-ahead keeps computation light for the onboard processor.
- Offline planning is unsafe because it assumes a map that does not exist; pure uninformed online search is too wasteful; hence heuristic online search is the best balance.

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

Answer:

CSP formulation (X, D, C).

- **Variables (X):** for each course/exam e, a start time slot T_e and a room (hall) R_e (and optionally an invigilator/faculty F_e). Example: X = {T_e, R_e, F_e for every exam e}.
- **Domains (D):** T_e ∈ {all available exam slots in the exam period, e.g. 10 days × 2 sessions = 20 slots}; R_e ∈ {Hall A, Hall B, …} with enough capacity; F_e ∈ {faculty members qualified for the subject}. Domains can be reduced beforehand by unary constraints.

Constraints (C):

- **No student clash (binary):** if two exams e1 and e2 share at least one student, T_e1 ≠ T_e2.
- **Hall capacity / limited halls:** two exams in the same slot cannot use the same hall (if T_e1 = T_e2 then R_e1 ≠ R_e2), and the hall capacity must be at least the number of students (unary / binary).
- **Precedence:** if subject A must be before subject B, then T_A < T_B (ordering constraint).
- **Faculty availability:** T_e must be in the faculty's available slots (unary), and one faculty member cannot invigilate two exams in the same slot (binary).
- **Special accommodations:** for students needing extra time or a separate room, the slot must allow extended duration and a suitable room (unary and higher-order constraints), for example no consecutive exams for those students.
- **Soft constraints (optional):** spread the exams of a student across days, avoid very early or late slots; handled as an optimisation cost on top of the hard constraints.

The solution is a complete assignment of slot, hall and invigilator to every exam that satisfies all hard constraints. It can be represented as a constraint graph in which nodes are exams and edges join conflicting exams (similar to graph colouring where slots are colours).

ii. Explain how backtracking search works in this scenario

Answer:

Backtracking search. It assigns variables one at a time (depth-first) and checks the constraints after every assignment.

- Pick an unassigned exam, try a value (slot and hall) from its domain, and check consistency with the already-assigned exams (clash, capacity, precedence, faculty).

- If consistent, recurse to the next exam; if no value is consistent, **backtrack** to the previous variable and try its next value.
- It stops when all exams are assigned (success) or all options are exhausted (no timetable exists).

Because it rejects a partial assignment as soon as a constraint is violated, it avoids exploring whole subtrees, which is far better than generate-and-test. Without heuristics it can still be exponential, and it may thrash (repeat the same failure deeper in the tree). It is improved by ordering heuristics: **MRV** (choose the exam with the fewest legal slots left, e.g. the one with the most conflicts), **degree heuristic** (the exam in the most constraints), and **LCV** (try the slot that rules out the fewest options for neighbours), plus smarter backtracking such as conflict-directed backjumping.

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

Answer:

Constraint propagation reduces domains before and during search so that failures are detected early.

- **Forward checking:** after assigning exam e to slot s, remove s (and the used hall) from the domains of every unassigned exam that conflicts with e (shared students, same hall, same faculty). If any domain becomes empty, backtrack immediately, before wasting effort on deeper levels. Example: when Maths is placed in slot 3, all subjects sharing students with Maths lose slot 3 from their domains.
- **Arc consistency (AC-3 / MAC):** goes further, removing values that have no supporting value in a neighbouring variable's domain, and propagates these removals (for precedence: if A must precede B and A is in slot 5, B's domain loses slots ≤ 5, which can in turn prune other exams).
- **Benefits:** it detects dead ends earlier, shrinks the search tree dramatically, works well with MRV (domain sizes are kept up to date), and often solves practical timetables with little or no backtracking. The cost is extra work per node, which is usually repaid by the reduced number of nodes.

iv. Analyze real-world challenges and justify the best strategy for scalability

Answer:

Real-world challenges.

- **Scale:** thousands of exams and students create very large constraint graphs and exponential worst-case search; the problem is NP-hard (like graph colouring).
- **Over-constrained / conflicting requirements:** a perfect solution may not exist (too few halls or too many accommodations), so soft constraints and preferences are needed. Data changes late (late registrations, a faculty member becomes unavailable), so re-scheduling must be fast and stable (minimal changes).
- **Fairness and quality:** students should not get several exams on one day, and special-accommodation requirements must be respected while hard constraints are never violated.

Recommended strategy: a hybrid. Model it as a CSP and solve it with backtracking enhanced with MRV/degree/LCV ordering, forward checking or MAC (arc consistency), which guarantees constraint satisfaction. Decompose the problem (by department, day or student groups) and use **min-conflicts local search** or Simulated Annealing / Genetic Algorithms to repair or improve a near-solution and to optimise soft constraints. Incremental re-solving, in which only the affected exams are unassigned when something changes, supports updates. This

gives completeness for hard constraints with good scalability and flexibility for real-life changes.

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

Answer:

Minimax model. Minimax models the game as a two-player, zero-sum adversarial search problem between MAX (the AI) and MIN (the human).

- **Formal elements:** initial state (current game state), PLAYER(s) (whose turn), ACTIONS(s) (legal moves), RESULT(s, a) (transition), TERMINAL-TEST(s) (win/loss/draw or depth limit) and UTILITY(s) (payoff, e.g. +1 win, 0 draw, −1 loss, or a heuristic score).
- **Game tree:** nodes are game states and edges are moves, alternating between MAX and MIN levels.
- **Rule:** MAX nodes take the maximum of their children's values, MIN nodes take the minimum: MINIMAX(s) = max over actions if MAX to move, min over actions if MIN to move. The AI therefore assumes that the human plays optimally to minimise the AI's score and chooses the move that gives the best guaranteed outcome (the best worst case).
- Time complexity is O(b^m) and space O(bm) with depth-first evaluation, where b is the branching factor and m the maximum depth. For a real-time strategy game, the AI uses depth-limited minimax with an evaluation function at the cut-off depth.

ii. Analyze the computational challenges of large game trees

Answer:

Computational challenges.

- **Exponential growth:** with branching factor b and depth d, about b^d nodes must be examined. In a strategy game with many units, b can be in the hundreds or thousands (every unit may move, attack or build), so even depth 4–6 is infeasible. Chess has b ≈ 35 and depth ≈ 80, giving roughly 35^80 nodes.
- **Time limit:** decisions must be made within milliseconds to seconds, so full-depth search is impossible; a depth cut-off causes the horizon effect (dangerous events just beyond the search depth go unnoticed).
- **Real-time and imperfect information:** the game continues while the AI thinks, with simultaneous actions rather than strict turns, and hidden information (fog of war), which breaks the pure minimax assumptions.
- **Evaluation quality:** the approximate evaluation at the cut-off must be accurate; repeated states (transpositions) waste effort, and memory can also be a bottleneck.

These challenges motivate pruning, move ordering, depth limits, iterative deepening, transposition tables and abstraction of the action space.

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

Answer:

Alpha-Beta pruning. It returns exactly the same move as Minimax but skips branches that cannot influence the final decision.

- **α** = the best (highest) value found so far for MAX along the path to the root; **β** = the best (lowest) value found so far for MIN. The initial window is (α, β) = (−∞, +∞).
- **Pruning rule:** at any node, if α ≥ β the remaining children are not examined, because the opponent would never allow the game to reach this position. At a MIN node, if a child value ≤ α, MIN can already force a result no better than what MAX has available elsewhere, so MAX will never choose this branch (**α-cut**). At a MAX node, if a child value ≥ β, MIN already has a better alternative higher up, so the rest of the branch is irrelevant (**β-cut**).
- **Example:** MAX has explored move A and guaranteed a value of 5 (α = 5). In move B, MIN's first reply gives 3. Since MIN can hold MAX to ≤ 3, which is worse than 5, the other replies to B need not be evaluated.
- **Efficiency:** in the worst case (poor ordering) no pruning occurs and the complexity is O(b^d). With **perfect move ordering** it is O(b^(d/2)), so the search can look about twice as deep in the same time (or the effective branching factor falls from b to √b). With random ordering the complexity is about O(b^(3d/4)). Good ordering (captures and strong moves first, killer/history heuristics, iterative deepening using the previous iteration's best move) is therefore essential.
- **Result:** the same optimal decision as minimax with far fewer node evaluations, which is exactly what a time-limited game AI needs.

iv. Design an evaluation function and justify the factors considered

Answer:

Evaluation function. At the depth limit (or non-terminal state) the AI uses a heuristic estimate of the state value, usually a weighted linear sum: Eval(s) = w1·f1(s) + w2·f2(s) + … + wn·fn(s), where positive values favour the AI.

- **Material / army strength (f1):** (AI units × unit value) − (enemy units × unit value), including health, because losing units directly reduces fighting power.
- **Economy (f2):** resources, income rate, number of workers and bases; economic strength predicts long-term advantage.
- **Territory and map control (f3):** controlled area, key positions and chokepoints, and vision (information advantage).
- **Tech and production capacity (f4):** upgrades, production buildings, and ability to rebuild.
- **Safety / threat (f5):** distance of enemy forces to the base, vulnerable units, and unit positioning (formation, high ground).
- **Tempo (f6):** whose attack is imminent, and the timing advantage.

Justification: each factor is cheap to calculate (necessary for speed), measures something that is correlated with winning, and the factors complement each other. The weights are set from expert knowledge or learned from self-play or game data (for example by machine learning / TD-learning) and can change with game phase (early game favours economy, late game favours army). Terminal states return extreme values (±∞) for win/loss. The function should be fast, accurate, and applied only to quiescent (stable) positions to avoid the horizon effect.

v. Recommend a strategy for balancing optimality and real-time performance

Answer:

Recommended strategy: depth-limited, iterative-deepening Alpha-Beta search with a strong evaluation function and time management, supplemented by search-reduction techniques.

- **Iterative deepening with a time budget:** search depth 1, 2, 3, … until the time limit is reached, and play the best move from the last completed iteration. This always provides an answer (an anytime algorithm), and each iteration provides move ordering for the next.
- **Better pruning:** move ordering (best moves first), transposition tables to reuse the values of repeated states, killer/history heuristics, aspiration windows and null-move pruning.
- **Selective search:** forward pruning or beam search to consider only promising moves, abstraction of the action space (group units into squads, high-level strategic actions), and quiescence search to extend unstable positions.
- **Cutting the cost further:** use Monte Carlo Tree Search (MCTS) when the branching factor is huge or imperfect information is present, and precomputed opening books or scripted behaviours for common situations; run the search in a parallel thread so the game never stalls.
- **Trade-off:** the AI sacrifices guaranteed optimality for acceptable, time-bounded play. It adapts the depth and the pruning aggressiveness to the available time, and the search reaches deeper only where it matters, so it achieves strong, fast and robust decisions.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
		effectively to address the scenario			
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand